Università degli studi di Messina
Dipartimento di Scienze Matematiche, Informatiche, Scienze Fisiche e Scienze della Terra

Course: Date Analysis
Project on “System Security ”

Project name: “Fake Data Prevention with Conventional Cryptotools”

Student:
Kasymalieva Aimira(551022)

MESSINA 2026

Content

1. Introduction

In modern information systems, data security is a critical concern. Data transmitted between clients and servers can be intercepted, modified, or forged by attackers. Such attacks may lead to incorrect system behavior, data corruption, or even financial loss.

One of the most common threats is **data tampering**, where an attacker modifies the content of a message after it has been created by a legitimate sender. Without proper protection mechanisms, the server cannot distinguish between valid and manipulated data.

The main objective of this project is to design and implement a system that prevents fake or modified data from being accepted. The system uses conventional cryptographic tools such as:

- Digital signatures
- Public/private key cryptography
- JSON Web Tokens (JWT)
- Symmetric encryption (optional layer)

The focus of the project is on ensuring **data integrity**, **authenticity**, and **secure communication** between client and server.

2. System Architecture

The system follows a client-server architecture and consists of three main components:

1. Backend (FastAPI)

The backend is responsible for:

- handling user authentication
- verifying digital signatures
- decrypting data (in encrypted mode)
- logging requests

2. Client (Python scripts)

The client simulates a real device that:

- generates data (payload)
- signs the data using a private key
- optionally encrypts the data
- sends requests to the server

3. Database (SQLite)

The database stores:

- user information
- public keys
- logs of all incoming data

This architecture reflects real-world systems where devices send data to a centralized server.

3. Authentication Mechanism

The system uses **JWT (JSON Web Token)** to authenticate users.

Registration Process

During registration:

- a new user is created
- a public/private key pair is generated
- the **public key is stored on the server**
- the **private key is returned to the client**

This step establishes the cryptographic identity of the user.

```python
@router.post("/register", response_model=RegisterResponse)
def register_user(user_data: UserRegister, db: Session = Depends(get_db)):
    existing_username = db.query(User).filter(User.username == user_data.username).first()
    if existing_username:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Username already exists"
        )

    existing_email = db.query(User).filter(User.email == user_data.email).first()
    if existing_email:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Email already exists"
        )

    private_key, public_key = generate_keys()

    new_user = User(
        username=user_data.username,
        email=user_data.email,
        password_hash=hash_password(user_data.password),
        public_key=public_key
    )

    db.add(new_user)
    db.commit()
    db.refresh(new_user)

    return {
        "message": "User registered successfully",
        "private_key": private_key
    }
```

Login Process

During login:

- the user provides username and password
- the server verifies credentials
- a JWT token is generated

This token is used in future requests.

```python
@router.post("/login", response_model=TokenResponse)
def login_user(user_data: UserLogin, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.username == user_data.username).first()

    if not user:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid username or password"
        )

    if not verify_password(user_data.password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid username or password"
        )

    token = create_access_token(
        data={
            "sub": user.username,
            "user_id": user.id
        }
    )

    return {
        "access_token": token,
        "token_type": "bearer"
    }
```

4. Digital Signature

Digital signature is the core mechanism used in this project.

How it works

1. The client creates a payload (data)
2. The payload is converted into a structured format (JSON)
3. A hash of the payload is generated
4. The hash is signed using the **private key**

The server:

- retrieves the corresponding **public key**
- verifies the signature

```python
def sign_data(private_key_pem: str, data: bytes) -> bytes:
    private_key = serialization.load_pem_private_key(
        PRIVATE_KEY.encode("utf-8"),
        password=None
    )

    signature = private_key.sign(
        data,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )
    return signature
```

```python
is_valid = verify_signature(user.public_key, data_bytes, signature_bytes)
```

5. Payload Structure

The payload represents the actual data sent to the server.

Example:

```
{
  "device_id": "sensor_01",
  "temperature": 25,
  "humidity": 60
}
```

This data simulates real-world sensor data.

6. Attack Simulation

To validate the effectiveness of the proposed security mechanism, an attack scenario was intentionally simulated within the system. The goal of this simulation is to demonstrate how the system behaves when data integrity is compromised.

Attack Scenario Description

In a normal scenario, the client generates a payload and signs it using the private key. The signature is mathematically linked to the exact content of the payload. This ensures that any modification of the data will invalidate the signature.

In the simulated attack:

1. The client generates a valid payload (e.g., temperature = 25)
2. The payload is signed using the private key
3. After signing, the payload is intentionally modified (e.g., temperature = 999)
4. The original signature is reused and sent with the modified payload

This mimics a real-world attack where an attacker intercepts the message and modifies its content without having access to the private key.

```
original_payload = {
    "device_id": "sensor_01",
    "temperature": 25,
    "humidity": 60,
    "timestamp": "2026-04-22T23:20:00"
}

original_json = json.dumps(
    original_payload,
    sort_keys=True,
    separators=(",", ":")
)
original_bytes = original_json.encode("utf-8")

signature = sign_data(PRIVATE_KEY, original_bytes)
signature_b64 = base64.b64encode(signature).decode("utf-8")

headers = {
    "Authorization": f"Bearer {TOKEN}"
}

print("=== VALID REQUEST ===")
response_valid = requests.post(
    URL,
    json={
        "payload": original_payload,
        "signature": signature_b64
    },
    headers=headers
)

print("Status:", response_valid.status_code)
print("Response:", response_valid.text)
print()
```

```
tampered_payload = {
    "device_id": "sensor_01",
    "temperature": 999,
    "humidity": 60,
    "timestamp": "2026-04-22T23:20:00"
}

print("=== TAMPERED REQUEST ===")
response_tampered = requests.post(
    URL,
    json={
        "payload": tampered_payload,
        "signature": signature_b64
    },
    headers=headers
)

print("Status:", response_tampered.status_code)
print("Response:", response_tampered.text)
```

Result

•The server detects mismatch between payload and signature
•The request is rejected

```
(venv) (base) ajmirakasymalieva@Ajmiras-MacBook-Air fake-data-prevention % /Users/ajm
bin/python /Users/ajmirakasymalieva/Desktop/fake-data-prevention/client/sender.py
=== VALID REQUEST ===
Status: 200
Response: {"message":"Data accepted (valid signature)"}

=== TAMPERED REQUEST ===
Status: 400
Response: {"detail":"Fake or tampered data detected"}
```

7. Logging and Audit Trail

An important extension of the system is the implementation of a logging mechanism that records all incoming requests, including both valid and invalid ones.

Logging Structure

Each log entry contains the following information:

- **user_id** – identifies the sender
- **payload** – the received data
- **is_valid** – indicates whether the signature was valid
- **timestamp** – time of the request

```python
class DataLog(Base):
    __tablename__ = "data_logs"

    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    payload = Column(String)
    is_valid = Column(Boolean)  # True = valid, False = tampered
    created_at = Column(DateTime, default=datetime.utcnow)
```

```python
log = DataLog(
    user_id=user.id,
    payload=json.dumps(request.payload),
    is_valid=is_valid
)
db.add(log)
db.commit()
```

Purpose of Logging

The logging system serves several important functions:

1. Monitoring System Activity

It allows tracking all interactions with the system, including normal usage and suspicious attempts.

2. Detection of Attacks

Invalid or tampered requests are recorded, making it possible to detect patterns of malicious behavior.

3. Audit Trail

The logs provide a complete history of actions, which is essential for:

- debugging
- incident analysis
- forensic investigation

8. Encryption Layer

In addition to digital signatures, the system includes an optional encryption layer to provide data confidentiality.

Motivation

While digital signatures ensure that data has not been modified, they do not prevent unauthorized parties from reading the data. Therefore, encryption is introduced to protect sensitive information during transmission.

Encryption Process

The system uses symmetric encryption (Fernet) for simplicity and efficiency.

The process is as follows:

On the client side:

1. The payload is converted into a JSON string
2. The payload is encrypted using a shared encryption key
3. The encrypted payload is then signed using the private key
4. Both encrypted data and signature are sent to the server

On the server side:

1. The signature is verified using the public key
2. If valid, the encrypted payload is decrypted
3. The original data is restored and processed

```python
def encrypt_payload(payload: dict) -> str:
    payload_json = json.dumps(
        payload,
        sort_keys=True,
        separators=(",", ":")
    )

    fernet = Fernet(DEMO_ENCRYPTION_KEY)
    encrypted = fernet.encrypt(payload_json.encode("utf-8"))

    return encrypted.decode("utf-8")
```

```python
def encrypt_data(plain_text: str) -> str:
    fernet = Fernet(DEMO_ENCRYPTION_KEY)
    encrypted = fernet.encrypt(plain_text.encode("utf-8"))
    return encrypted.decode("utf-8")


def decrypt_data(encrypted_text: str) -> str:
    fernet = Fernet(DEMO_ENCRYPTION_KEY)
    decrypted = fernet.decrypt(encrypted_text.encode("utf-8"))
    return decrypted.decode("utf-8")
```

9. Conclusion

This project presents a practical implementation of a secure data transmission system using conventional cryptographic tools.

The system successfully demonstrates how multiple security mechanisms can be combined to protect data from unauthorized modification and misuse.

The system ensures:

- **Authentication** using JWT, allowing only authorized users to send data
- **Integrity** through digital signatures, ensuring that data has not been altered
- **Authenticity** by verifying that data originates from a legitimate user
- **Confidentiality** through optional encryption of payloads
- **Traceability** via logging and audit trails

The implemented approach effectively prevents common attack scenarios such as:

- data tampering
- unauthorized data injection
- replay of modified data

The system detects and rejects invalid requests while maintaining a record of all interactions.

Overall, the project demonstrates that even relatively simple cryptographic techniques can significantly improve system security.

By combining authentication, digital signatures, encryption, and logging, the system provides a robust foundation for secure data communication.